UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

S2

Problemas y espacios de búsqueda

Actividad Parcial 2 · Fundamentos de Inteligencia Artificial

Temas 2.1 y 2.2 · Modelado, grafos y complejidad

Alumno	Rogerio Aguilar Bello
Matrícula	24305118
Grupo / cuatrimestre	171 · 7.º
Docente	Mtro. Javier García Lara
Tipo de entrega	Individual · 50 pts
Fecha de entrega	18 de octubre de 2026
Referencia del tema	Bohm (2021), caps. 1–2
Repositorio GitHub	[https://github.com/usuario/FIA-portafolio]

Capítulo 1. Planteamiento del problema

1.1 Contexto

Antes de aplicar un algoritmo de búsqueda, hay que **formular** el problema: decidir qué es un estado, qué acciones existen, cuánto cuesta cada una y cómo se reconoce la meta. Una formulación deficiente produce espacios de búsqueda enormes o soluciones incorrectas, aunque el algoritmo sea bueno.

1.2 Problema

Dados dos problemas reales (ruteo de una unidad de reparto y navegación de un robot de almacén), ¿cómo modelarlos como problemas de búsqueda, representar su espacio de estados como grafo y estimar la complejidad del espacio que deberá explorar un algoritmo?

1.3 Datos de los problemas

Problema A (reparto). Una unidad sale del depósito D, debe visitar 4 clientes (A, B, C, E) una vez cada uno y regresar a D. Los tiempos de viaje entre ubicaciones (10 a 25 min) son **datos ilustrativos** definidos para el ejercicio, no mediciones; en un caso productivo se obtendrían de GPS o de una API de mapas. **Problema B (almacén).** Un robot parte de la celda S de una cuadrícula de 5 × 6 con 7 muros y 2 celdas de piso lento y debe llegar a la celda de entrega G; moverse a una celda normal cuesta 1 y a una celda lenta cuesta 3.

Capítulo 2. Objetivos

2.1 Objetivo general

Modelar dos problemas reales como problemas de búsqueda **mediante** la definición formal de estados, acciones, costo y meta y su representación como grafos con NetworkX **para** estimar la complejidad de su espacio de estados antes de elegir un algoritmo de búsqueda.

2.2 Objetivos específicos

- **Definir** estado inicial, acciones, modelo de transición, costo y prueba de meta de cada problema **aplicando** la formulación estándar **para** eliminar ambigüedades en el modelado.
- **Programar** un generador de estados y un grafo dirigido ponderado con NetworkX **recorriendo** todos los estados alcanzables **para** obtener el espacio de búsqueda completo.
- **Calcular** el número de estados, el factor de ramificación y la profundidad de cada espacio **mediante** el análisis del grafo generado **para** estimar la complejidad.
- **Contrastar** el tamaño del espacio como grafo y como árbol de búsqueda **mediante** fórmulas y una tabla de escalamiento **para** justificar el control de estados repetidos.

Capítulo 3. Fundamentos

Un **problema de búsqueda** se define formalmente con cinco componentes (Russell y Norvig, 2020; Bohm, 2021): (1) el *estado inicial*; (2) las *acciones* aplicables en cada estado; (3) el *modelo de transición*, que dice a qué estado conduce una acción; (4) la *prueba de meta*; y (5) el *costo de camino*, suma de los costos de las acciones. Una **solución** es una secuencia de acciones del estado inicial a una meta; es **óptima** si su costo es mínimo.

El **espacio de estados** es el conjunto de estados alcanzables desde el inicial. Se representa como un grafo dirigido: los nodos son estados y las aristas, acciones, con el costo como peso. El **árbol de búsqueda** despliega

ese grafo en caminos; si el grafo tiene ciclos o caminos distintos hacia un mismo estado, el árbol repite estados y crece mucho más.

Medidas de complejidad: b = factor de ramificación (acciones por estado); d = profundidad de la meta menos profunda; $|S|$ = número de estados. Un árbol sin control de repetidos genera del orden de b^d nodos; un grafo con control de repetidos, a lo sumo $|S|$.

Capítulo 4. Metodología y procedimiento

- Describir cada problema real y sus datos de entrada.
- Formular los cinco componentes del problema de búsqueda y justificar la representación del estado.
- Implementar en Python la función de *sucesores* (generador de acciones) y la prueba de meta.
- Construir el grafo dirigido ponderado de NetworkX recorriendo el espacio en anchura desde el estado inicial.
- Medir $|S|$, número de aristas, b promedio, b máximo y profundidades; validar $|S|$ contra una fórmula.
- Validar el modelo obteniendo una solución óptima con Dijkstra y comparar el crecimiento del espacio como grafo y como árbol.
- Generar los diagramas de los grafos y redactar el análisis.

Capítulo 5. Modelado del Problema A: ruteo de reparto

Estado	(ubicación actual, conjunto de paradas visitadas). Ejemplo: (B, {A, B}). Se guarda el conjunto de visitadas porque la meta depende de ellas, no solo de la posición.
Estado inicial	(D, ∅): en el depósito, sin paradas visitadas.
Acciones	ir a(p) para cada parada p aún no visitada; si ya se visitaron todas, volver a D.
Modelo de transición	ir a(p) desde (x, V) conduce a (p, V ∪ {p}); volver a D conduce a (D, V).
Costo	Minutos de viaje entre las dos ubicaciones (ilustrativos). Costo de camino = suma de tiempos.
Prueba de meta	Ubicación = D y las 4 paradas visitadas.
Tipo de grafo	Dirigido (las acciones solo avanzan) y ponderado, sin ciclos (cada acción agrega una parada visitada).

Complejidad del espacio (k paradas)

- **Estados:** inicio (1) + pares (parada actual, subconjunto que la contiene) = $k \cdot 2^{k-1}$ + estado meta (1) ⇒ $|S| = k \cdot 2^{k-1} + 2$. Para k = 4: $4 \cdot 8 + 2 = 34$.
- **Factor de ramificación:** b = k en el primer paso y disminuye en uno por nivel; $b_{máx} = 4$.
- **Profundidad de la meta:** d = k + 1 (k visitas y el regreso) = 5.
- **Árbol sin control de repetidos:** k! secuencias de visita (24 para k = 4). Con k = 20, el grafo tiene 10,485,762 estados y el árbol 2.4×10^{18} secuencias: el espacio crece de forma exponencial, por lo que el control de estados repetidos y las heurísticas son indispensables.
- **Vínculo con la sección 2.4/2.5:** este problema es un agente viajero (TSP); su espacio no se puede recorrer por completo cuando k crece.

Capítulo 6. Modelado del Problema B: robot de almacén

Estado	(fila, columna) de la celda donde está el robot.
Estado inicial	(0, 0), celda S.

Acciones	arriba, abajo, izquierda, derecha, si la celda destino existe y no es muro.
Modelo de transición	La acción mueve al robot a la celda vecina correspondiente.
Costo	1 si la celda destino es pasillo; 3 si es zona lenta (piso mojado o de carga).
Prueba de meta	Posición = (4, 5), celda G.
Tipo de grafo	Dirigido y ponderado; cada par de celdas vecinas tiene aristas en ambos sentidos, así que hay ciclos.

Complejidad del espacio (cuadrícula R × C)

- **Estados:** $|S|$ = celdas libres alcanzables = 23 (verificado con el script; la cuadrícula tiene 30 celdas y 7 muros).
- **Factor de ramificación:** $b_{máx}$ = 3 y b promedio = 2.26 (52 transiciones entre 23 estados), por los muros y los bordes.
- **Profundidad de la meta:** d = 9 acciones.
- **Árbol sin control de repetidos:** como el robot puede retroceder, el árbol es infinito; sin evitar repetidos, la búsqueda puede ciclar. El grafo está acotado por R × C estados, así que el control de repetidos reduce un espacio potencialmente infinito a 23 nodos.
- **Escalamiento:** $|S| \leq R \times C$ y $b \leq 4$, de modo que el espacio crece de forma polinomial (cuadrática en el lado de la cuadrícula), mucho más manejable que el Problema A.

Capítulo 7. Resultados: diagramas y tablas

7.1 Diagrama del grafo del Problema A

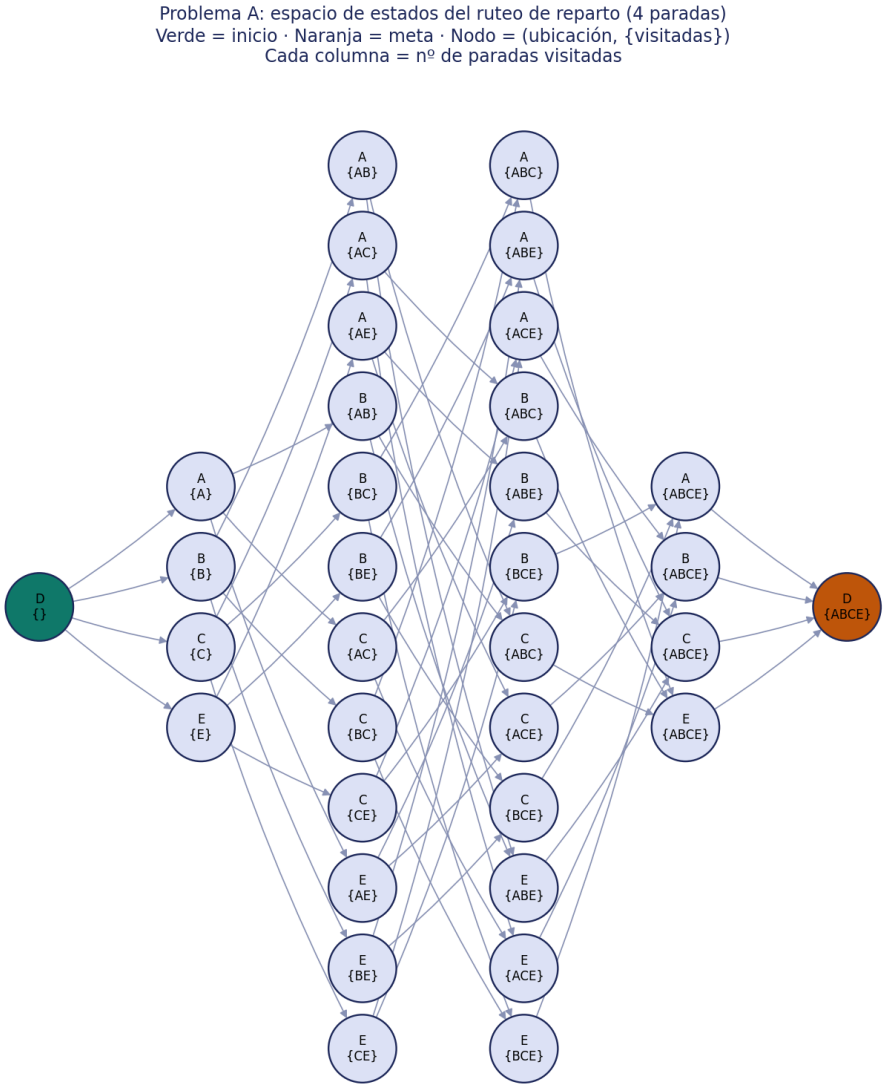


Figura 1. Espacio de estados del ruteo de reparto (34 nodos, 56 aristas). Los pesos de las aristas son los tiempos de viaje del código. Elaboración propia con NetworkX y matplotlib.

7.2 Diagrama del grafo del Problema B

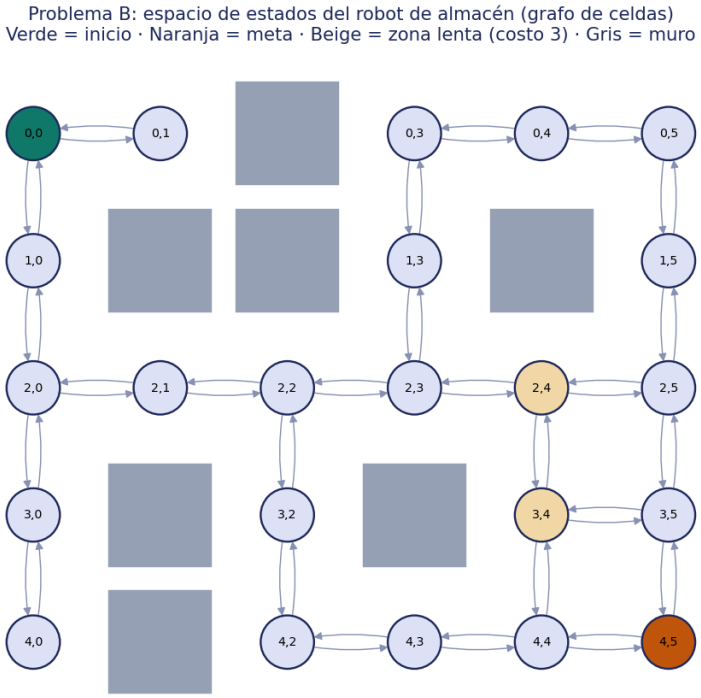


Figura 2. Espacio de estados del robot de almacén (23 nodos, 52 aristas dirigidas). Elaboración propia con NetworkX y matplotlib.

7.3 Tabla comparativa de modelado y complejidad

Criterio	Problema A: reparto	Problema B: almacén
Estado	(ubicación, conjunto de visitadas)	(fila, columna)
Estados |S|	34 (= $k \cdot 2^{k-1} + 2$, k = 4)	23 (celdas libres)
Transiciones	56	52
b promedio / máximo	1.65 / 4	2.26 / 3
Profundidad de la meta d	5	9
¿Hay ciclos?	No (grafo acíclico)	Sí (se puede retroceder)
Crecimiento con el tamaño	Exponencial: $k \cdot 2^{k-1}$ estados; k! secuencias	Polinomial: ≤ R × C estados
Solución óptima (Dijkstra)	D ⟶ A ⟶ B ⟶ C ⟶ E ⟶ D, 57 min	9 movimientos, costo 9

7.4 Escalamiento del Problema A

k paradas	Estados (grafo)	Secuencias (árbol, k!)	Razón árbol / grafo
4	34	24	0.7×
8	1,026	40,320	39.3×
12	24,578	479,001,600	19,489×
16	524,290	2.09×10^{13}	3.99×10^{7}×
20	10,485,762	2.43×10^{18}	2.32×10^{11}×

Con k = 4 el árbol es incluso más pequeño que el grafo porque el grafo incluye estados intermedios; a partir de k ≈ 5 el árbol supera al grafo y la diferencia crece de forma explosiva.

7.5 Salida del script

```
[A reparto] estados=34 transiciones=56 b_prom=1.65 b_max=4 profundidad_meta=5 profundidad_max=5 estados_meta=1
[B almacén] estados=23 transiciones=52 b_prom=2.26 b_max=3 profundidad_meta=9 profundidad_max=9 estados_meta=1
Verificación A: estados reales=34 vs fórmula 1+k*2^(k-1)+1=34
Verificación B: estados=23 vs celdas libres=23

 k  | estados (grafo) | secuencias (arbol, k!) | razon
  4 |              34 |                     24 | 0.7x
  8 |           1,026 |                 40,320 | 39.3x
 12 |          24,578 |            479,001,600 | 19,489.0x
 16 |         524,290 |     20,922,789,888,000 | 39,906,902.5x
 20 |      10,485,762 | 2,432,902,008,176,640,000 | 232,019,571,698.9x

Ruta óptima A: D -> A -> B -> C -> E -> D | costo = 57 min
Ruta óptima B: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)] | costo = 9
```

Capítulo 8. Código Python

Script reproducible (`fia_s2_espacios_busqueda.py`). Requisitos: `pip install networkx matplotlib`. Se ejecutó con los datos de entrada definidos en el propio código y su salida se muestra en la sección 7.5. Se usó NetworkX (Hagberg et al., 2008) y matplotlib (Hunter, 2007).

```
"""
FIA - Actividad Parcial 2 - Sección 2 (2.1 Problemas de búsqueda, 2.2 Espacios de búsqueda)

Modela DOS problemas como problemas de búsqueda (estado inicial, acciones,
modelo de transición, costo y prueba de meta), construye su espacio de
estados como grafo dirigido y ponderado con NetworkX, genera los diagramas
y calcula la complejidad del espacio.

  Problema A: ruteo de una unidad de reparto (visitar todas las paradas y regresar).
  Problema B: robot de almacén en una cuadrícula con obstáculos y zonas lentas.

Requisitos: pip install networkx matplotlib
Uso:        python fia_s2_espacios_busqueda.py
"""
from itertools import count
from math import factorial
import networkx as nx
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


# ============================================================================
# PROBLEMA A: RUTEO DE REPARTO
# Estado   : (ubicacion_actual, paradas_visitadas)  -> paradas_visitadas es frozenset
# Acciones : ir a una parada aun no visitada; con todas visitadas, volver al depósito
# Costo    : minutos de viaje entre ubicaciones (DATOS ILUSTRATIVOS, no medidos)
# Meta     : (depósito, todas las paradas visitadas)
# ============================================================================
DEPOSITO = "D"
PARADAS = ["A", "B", "C", "E"]  # 4 clientes
TIEMPOS = {  # minutos, simétricos; reemplazar con datos reales (API de mapas / GPS)
    ("D", "A"): 12, ("D", "B"): 20, ("D", "C"): 25, ("D", "E"): 15,
    ("A", "B"): 10, ("A", "C"): 18, ("A", "E"): 14,
    ("B", "C"): 9,  ("B", "E"): 22,
    ("C", "E"): 11,
}


def tiempo(u, v):
    return TIEMPOS.get((u, v), TIEMPOS.get((v, u)))


def estado_inicial_A():
    return (DEPOSITO, frozenset())


def es_meta_A(s):
    loc, vis = s
    return loc == DEPOSITO and len(vis) == len(PARADAS)


def acciones_A(s):
    """Generador de sucesores: devuelve (accion, nuevo_estado, costo)."""
    loc, vis = s
    pendientes = [p for p in PARADAS if p not in vis]
    if pendientes:
        for p in pendientes:
            yield (f"ir a {p}", (p, vis | {p}), tiempo(loc, p))
    elif loc != DEPOSITO:
        yield ("volver a D", (DEPOSITO, vis), tiempo(loc, DEPOSITO))


def generar_espacio(estado_ini, sucesores):
    """Recorre (BFS) todos los estados alcanzables y arma el grafo dirigido ponderado."""
    G = nx.DiGraph()
    G.add_node(estado_ini)
    frontera = [estado_ini]
```

```
    while frontera:
        s = frontera.pop(0)
        for accion, s2, costo in sucesores(s):
            if s2 not in G:
                G.add_node(s2)
                frontera.append(s2)
            G.add_edge(s, s2, weight=costo, accion=accion)
    return G


def etiqueta_A(s):
    loc, vis = s
    return f"{loc}\n{{{''.join(sorted(vis))}}}"


# ============================================================================
# PROBLEMA B: ROBOT DE ALMACÉN
# Estado   : (fila, columna)
# Acciones : arriba, abajo, izquierda, derecha (si la celda existe y no es muro)
# Costo    : 1 en pasillo normal; 3 en zona lenta (piso mojado/carga)
# Meta     : celda de entrega
# ============================================================================
MAPA = [
    "S.#...",
    ".##.#.",
    "....~.",
    ".#.#~.",
    ".#...G",
]  # S=inicio, G=meta, #=muro, ~=zona lenta (costo 3), .=pasillo (costo 1)
MOV = {"arriba": (-1, 0), "abajo": (1, 0), "izquierda": (0, -1), "derecha": (0, 1)}


def buscar(simbolo):
    for r, fila in enumerate(MAPA):
        if simbolo in fila:
            return (r, fila.index(simbolo))


INICIO_B, META_B = buscar("S"), buscar("G")


def es_meta_B(s):
    return s == META_B


def acciones_B(s):
    r, c = s
    for nombre, (dr, dc) in MOV.items():
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(MAPA) and 0 <= nc < len(MAPA[0]) and MAPA[nr][nc] != "#":
            yield (nombre, (nr, nc), 3 if MAPA[nr][nc] == "~" else 1)


# ============================================================================
# COMPLEJIDAD
# ============================================================================
def analisis(G, sucesores, nombre, es_meta, estado_ini):
    """Mide |S|, |A|, factor de ramificación b y profundidad máxima del grafo."""
    n, m = G.number_of_nodes(), G.number_of_edges()
    b_prom = m / n
    b_max = max(d for _, d in G.out_degree())
    metas = [s for s in G if es_meta(s)]
    prof = nx.single_source_shortest_path_length(G, estado_ini)
    d_meta = min(prof[s] for s in metas)
    d_max = max(prof.values())
    print(f"[{nombre}] estados={n} transiciones={m} b_prom={b_prom:.2f} b_max={b_max} "
          f"profundidad_meta={d_meta} profundidad_max={d_max} estados_meta={len(metas)}")
    return dict(n=n, m=m, b_prom=b_prom, b_max=b_max, d_meta=d_meta, d_max=d_max)


def estados_A_formula(k):
    """|S| teórico: inicio + (parada_actual, subconjunto con ella) + estado meta."""
    return 1 + k * 2 ** (k - 1) + 1


def tabla_escalamiento():
    print("\n k  | estados (grafo) | secuencias (arbol, k!) | razon")
```

```
    for k in (4, 8, 12, 16, 20):
        s, a = estados_A_formula(k), factorial(k)
        print(f"{k:>3} | {s:>15,} | {a:>22,} | {a / s:,.1f}x")


# ============================================================================
# DIAGRAMAS
# ============================================================================
def dibujar_A(G, ruta="grafo_problema_A.png"):
    # Capas por número de paradas visitadas (nivel) para ordenar el dibujo
    niveles = {}
    for s in G:
        niv = len(s[1]) + (1 if es_meta_A(s) else 0)
        niveles.setdefault(niv, []).append(s)
    pos = {}
    for niv, nodos in niveles.items():
        nodos.sort(key=lambda s: (s[0], sorted(s[1])))
        for i, s in enumerate(nodos):
            pos[s] = (niv * 2.0, (len(nodos) - 1) / 2 * 1.35 - i * 1.35)
    colores = ["#0F7B6C" if s == estado_inicial_A() else "#C2570C" if es_meta_A(s) else "#DDE3F5" for s in G]
    fig, ax = plt.subplots(figsize=(10, 11), dpi=170)
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#8A93B5", arrows=True, arrowsize=9, width=0.7,
                           connectionstyle="arc3,rad=0.05", node_size=1500)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_color=colores, node_size=1500, edgecolors="#1F2A5C")
    nx.draw_networkx_labels(G, pos, {s: etiqueta_A(s) for s in G}, font_size=7, ax=ax)
    ax.set_title("Problema A: espacio de estados del ruteo de reparto (4 paradas)\n"
                 "Verde = inicio · Naranja = meta · Nodo = (ubicación, {visitadas})\n"
                 "Cada columna = nº de paradas visitadas", fontsize=10, color="#1F2A5C")
    ax.axis("off")
    fig.savefig(ruta, bbox_inches="tight"); plt.close(fig)


def dibujar_B(G, ruta="grafo_problema_B.png"):
    pos = {s: (s[1], -s[0]) for s in G}  # la celda (fila, col) se dibuja donde está
    colores = ["#0F7B6C" if s == INICIO_B else "#C2570C" if s == META_B
               else "#F2D9A7" if MAPA[s[0]][s[1]] == "~" else "#DDE3F5" for s in G]
    fig, ax = plt.subplots(figsize=(9, 6), dpi=170)
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#8A93B5", arrows=True, arrowsize=8, width=0.6,
                           connectionstyle="arc3,rad=0.12", node_size=700)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_color=colores, node_size=700, edgecolors="#1F2A5C")
    nx.draw_networkx_labels(G, pos, {s: f"{s[0]},{s[1]}" for s in G}, font_size=6, ax=ax)
    for r, fila in enumerate(MAPA):  # muros dibujados como cuadros grises
        for c, ch in enumerate(fila):
            if ch == "#":
                ax.add_patch(plt.Rectangle((c - .4, -r - .4), .8, .8, color="#9AA0B4"))
    ax.set_title("Problema B: espacio de estados del robot de almacén (grafo de celdas)\n"
                 "Verde = inicio · Naranja = meta · Beige = zona lenta (costo 3) · Gris = muro",
                 fontsize=9, color="#1F2A5C")
    ax.set_aspect("equal"); ax.axis("off")
    fig.savefig(ruta, bbox_inches="tight"); plt.close(fig)


# ============================================================================
# EJECUCIÓN
# ============================================================================
if __name__ == "__main__":
    GA = generar_espacio(estado_inicial_A(), acciones_A)
    GB = generar_espacio(INICIO_B, acciones_B)

    ra = analisis(GA, acciones_A, "A reparto", es_meta_A, estado_inicial_A())
    rb = analisis(GB, acciones_B, "B almacén", es_meta_B, INICIO_B)
    print(f"Verificación A: estados reales={ra['n']} vs fórmula 1+k*2^(k-1)+1={estados_A_formula(len(PARADAS))}")
    celdas_libres = sum(f.count(".") + f.count("~") + f.count("S") + f.count("G") for f in MAPA)
    print(f"Verificación B: estados={rb['n']} vs celdas libres={celdas_libres}")
    tabla_escalamiento()

    # Solución óptima (costo uniforme con Dijkstra de NetworkX) para validar el modelo
    metaA = next(s for s in GA if es_meta_A(s))
    rutaA = nx.dijkstra_path(GA, estado_inicial_A(), metaA, weight="weight")
    costoA = nx.dijkstra_path_length(GA, estado_inicial_A(), metaA, weight="weight")
    print("\nRuta óptima A:", " -> ".join(s[0] for s in rutaA), f"| costo = {costoA} min")
    rutaB = nx.dijkstra_path(GB, INICIO_B, META_B, weight="weight")
    costoB = nx.dijkstra_path_length(GB, INICIO_B, META_B, weight="weight")
    print("Ruta óptima B:", rutaB, f"| costo = {costoB}")

    dibujar_A(GA); dibujar_B(GB)
    print("Diagramas: grafo_problema_A.png, grafo_problema_B.png")
```

Capítulo 9. Análisis de complejidad y discusión

9.1 Complejidad del espacio de estados

El Problema A tiene un espacio exponencial en el número de paradas: $|S| = k \cdot 2^{k-1} + 2$ y $k!$ caminos. Es el comportamiento típico de un problema combinatorio (agente viajero) y explica por qué se necesitan heurísticas y búsqueda local en las secciones siguientes. El Problema B tiene un espacio polinomial acotado por $R \times C$; es un problema "fácil" para búsqueda en grafos, y su dificultad principal es evitar ciclos.

9.2 Efecto de la representación del estado

Si en el Problema A el estado fuera solo la ubicación, el modelo perdería información: llegar a B habiendo visitado {A, B} es distinto de llegar a B habiendo visitado {A, B, C}, y la prueba de meta no podría evaluarse. Un estado mínimo pero suficiente evita tanto la pérdida de información como el crecimiento innecesario del espacio.

9.3 Control de estados repetidos

En el Problema B, el grafo tiene 23 nodos pero el árbol de búsqueda es infinito si no se descartan estados ya visitados. En el Problema A, aunque no hay ciclos, distintos órdenes de visita llegan al mismo estado (por ejemplo, (C, {A, B, C}) se alcanza por dos caminos), y fusionarlos es lo que reduce $k!$ a $k \cdot 2^{k-1}$.

9.4 Validación del modelo

Dijkstra sobre el grafo generado devuelve soluciones óptimas coherentes con el modelo (57 min en A; costo 9 en B). Se comprobó que el número de estados del grafo coincide con la fórmula (34) y con las celdas libres (23), lo que respalda que el generador es correcto. Los tiempos de A son ilustrativos: el modelo es válido, pero la ruta óptima solo lo sería con datos reales.

9.5 Limitaciones

- El Problema A usa tiempos simétricos y constantes; en la realidad varían con el tráfico y la hora.
- No se modelaron restricciones adicionales (ventanas de entrega, capacidad de carga), que ampliarían el estado.
- El script genera el espacio completo; para k grande no es viable y se requeriría generar estados bajo demanda.

Capítulo 10. Conclusiones

- Formular bien el problema (estado, acciones, costo, meta) es el primer paso de cualquier búsqueda y determina el tamaño del espacio.
- Dos problemas reales de apariencia similar (rutas) tienen espacios muy distintos: exponencial el ruteo con múltiples paradas y polinomial la navegación en cuadrícula.
- Representar el espacio como grafo con NetworkX permite medir $|S|$, b y d, validar el modelo con fórmulas y comparar contra el árbol de búsqueda.
- El control de estados repetidos reduce drásticamente el espacio, pero no elimina el crecimiento exponencial del Problema A; eso motiva las heurísticas y la búsqueda local de las siguientes actividades.

Capítulo 11. Plus: explorador interactivo, representaciones y crecimiento

11.1 Explorador interactivo de espacios de búsqueda

Se desarrolló un archivo independiente, `explorador_espacios_busqueda.html`, que se abre con doble clic en cualquier navegador (sin instalación ni internet). Permite **recorrer a mano** el espacio de estados de cada problema y comparar el costo propio contra el óptimo:

- **Pestaña A (reparto):** en cada paso muestra el estado (ubicación, {visitadas}), las acciones disponibles con su costo y el costo acumulado. Calcula en vivo el óptimo (57 min, por fuerza bruta sobre 24 permutaciones) y $|S| = 34$ (recorrido en anchura).
- **Pestaña B (almacén):** el robot se mueve con botones o flechas del teclado; muestra el costo acumulado frente al óptimo (9, por Dijkstra) y $|S| = 23$.
- **Verificación:** la lógica del explorador se ejecutó en Node.js y reprodujo exactamente los valores del script de Python (57, 34, 9 y 23).

11.2 Comparación de representaciones de estado (Problema A)

La elección de qué guardar en el estado cambia el tamaño del espacio. Se contó con el generador de estados cada alternativa (script `fia_s2_plus_crecimiento.py`):

Representación	Estados	¿Sirve?	Razón
R1: solo ubicación	5	No	No distingue qué paradas faltan: no puede evaluar la meta ni el costo restante.
R2: (ubicación, conjunto de visitadas)	34	Sí (mínima)	Guarda exactamente lo necesario; fusiona órdenes de visita equivalentes. Es la usada en la actividad.
R3: (ubicación, secuencia ordenada)	89	Sí, redundante	Distingue el orden, que no cambia lo que falta; crea 24 estados meta distintos y duplica trabajo.

Conclusión: el estado debe ser *suficiente* (permitir decidir acciones y meta) y *mínimo* (no distinguir lo que no importa). R2 reduce el espacio frente a R3 en más de la mitad sin perder información.

11.3 Crecimiento del espacio de búsqueda

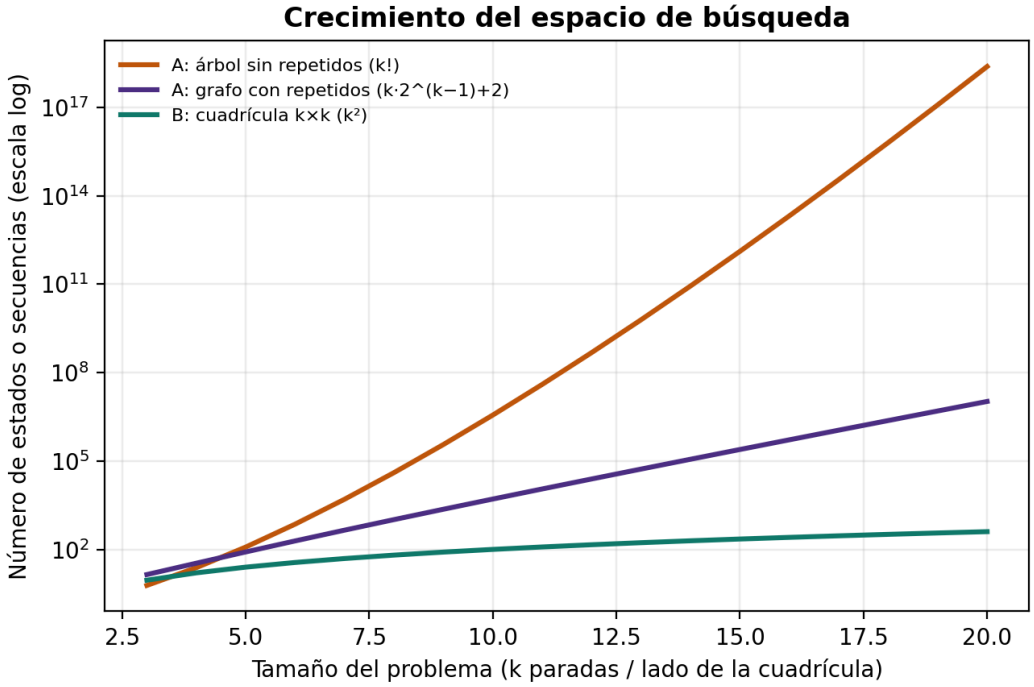


Figura 3. Estados o secuencias según el tamaño del problema (escala logarítmica). Elaboración propia con matplotlib.

La curva del árbol (k!) se separa de la del grafo desde k ≈ 5; la cuadrícula del Problema B crece solo de forma cuadrática. Esto muestra por qué un problema de ruteo con muchas paradas necesita heurísticas, mientras que la navegación en cuadrícula se resuelve con búsqueda en grafos.

Capítulo 12. Referencias (APA 7.ª ed.)

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Anexo A. Lista de cotejo (sección 2)

#	Criterio de evaluación	S2
1	Portada, tema, sección y datos del problema completos en el PDF	✓
2	Planteamiento, formulación y procedimiento claramente explicados	✓
3	Código Python funcional y comentado, probado con datos de entrada	✓
4	Solución enmarcada, con gráficos, tablas o diagramas de resultados	✓
5	Repositorio GitHub con carpeta del alumno, README y PDFs	✓
6	Declaración de IA utilizada, referencias APA 7 y conclusiones reflexivas	✓

Declaración de uso de IA

Herramienta: Claude (Anthropic), asistente conversacional.
Prompt: Ahora revisa el siguiente archivo y dale formato como el anterior, modifica la estructura según sea necesario y verifica que cumpla con la lista de cotejo, revisa las fuentes que sean confiables o modificarlas y genera un diagrama
Uso: estructurar el modelado, redactar el borrador, los diagramas y el diseño del documento.
Verificación: el código se ejecutó y su salida se incluyó en la sección 7.5; el número de estados se validó contra fórmulas y el costo de las rutas se comprobó a mano (57 min en A). Revisión personal del modelo y de las referencias por el alumno: [fecha].